

This assessment evaluates not only your ability to implement business features, but your capability to demonstrate true senior-level expertise. A senior developer is distinguished by their commitment to:

- **Reading official documentation thoroughly** before implementation
- **Making informed architectural decisions** based on authoritative sources
- **Understanding the "why" behind framework patterns** and best practices
- **Leading design decisions through human expertise** rather than AI suggestions

Documentation and personal experience must drive your decisions first, with AI tools serving only as supplementary assistance. Evidence of AI-driven development without proper documentation understanding will be considered a significant negative indicator during evaluation.

CRITICAL REQUIREMENT: CODE OWNERSHIP

You must fully understand and own every line of code you submit. After your submission, you will participate in a live technical session where you must:

- Walk through your entire codebase and explain every architectural decision
- Demonstrate a deep understanding of all patterns and frameworks used
- Answer detailed questions about your implementation choices
- Modify or extend your code in real-time during the discussion

AI tools are welcome, but creating AI-generated code without understanding will be immediately evident during the technical discussion.

Overview

Welcome to our technical assessment for the Senior Full Stack Team Lead position. This assignment is designed to evaluate your ability to architect and implement a modular Laravel application while demonstrating leadership-level technical skills.

Submission: Git repository with a complete working application

Assignment: E-commerce Order Management System

You will build a modular e-commerce system from scratch that demonstrates clean architecture, proper module boundaries, and modern Laravel development practices.

Technical Requirements

Technology Stack

- **Laravel**
- **nwidart/laravel-modules** package for module management
- **Livewire** for interactive frontend components
- **Filament** for admin interfaces
- **PostgreSQL** database
- **Pest** testing framework

Architecture Requirements

- **Modular Structure:** Use **nwidart/laravel-modules** to create separate modules
- **Module Independence:** Modules must NOT directly access each other's models or services
- **Cross-module Communication:** Implement proper interfaces/contracts for inter-module communication
- **Clean Boundaries:** Demonstrate understanding of module decoupling principles

Business Requirements

Module 1: Catalog Module

Purpose: Product management and catalog functionality

Features:

- Product entity with: name, description, price, stock quantity
- Product categories (simple relationship)
- Filament admin interface for product CRUD operations
- Public product browsing capability

Module 2: Order Module

Purpose: Order processing and management

Features:

- Order entity with: customer info, order items, total amount, status
- Order-Product relationship (without direct Catalog model dependency)
- Order status workflow (pending → confirmed → shipped → delivered)
- Filament admin interface for order management
- Livewire component for order creation/viewing

Core Challenge: Cross-Module Communication

The Key Architectural Test: Your Order module needs to work with Product data but CANNOT directly use `Catalog\Models\Product` or any Catalog services directly.

You must design and implement a solution that allows:

- Displaying available products during order creation
- Storing product information with orders
- Showing product details in order views
- Maintaining data consistency

Testing Requirements

Test Framework

- Use **Pest** testing framework
- Write **feature tests** (not unit tests) for main functionality
- Organize tests within each module's directory structure

Test Organization

None

```
Modules/  
├─ Catalog/  
│   └─ tests/  
│       └─ Feature/  
│           ├── ProductManagementTest.php  
│           └─ ProductDisplayTest.php  
├─ Order/  
│   └─ tests/  
│       └─ Feature/  
│           ├── OrderCreationTest.php  
│           └─ OrderManagementTest.php
```

Required Test Coverage

- Product CRUD operations
- Order creation workflow
- Cross-module functionality (how orders work with products)
- Admin interface interactions
- Livewire component behavior

Code Style & Quality Requirements

Code Formatting

- Use **Laravel Duster** for code formatting and style consistency
- Ensure all code passes style checks before submission

Quality Checks

Your code should pass:

- Style formatting (Laravel Duster)
- All tests (Pest)

Bonus: Static Analysis

Implementing static analysis demonstrates senior-level code quality practices:

- **Larastan** (Laravel-specific PHPStan) for type checking at least on level 5
- **Zero errors** on analysis level that you configure
- **Type hints** and proper Laravel code patterns
- **Integration** with your development workflow

Deliverables

1. Working Application

- Complete Laravel application with 2+ modules
- All specified functionality working
- Database migrations for all entities
- Seeded data for demonstration

2. Admin Interfaces

- Filament admin panels for both modules
- Intuitive product management interface
- Comprehensive order management interface

3. Frontend Components

- Livewire components for order creation
- Product browsing/selection interface
- Order status display

5. Comprehensive README

Your README must include:

Setup Instructions

Shell

```
# Installation steps
# Database setup
# Environment configuration
# Asset compilation
```

Running the Application

Shell

```
# How to start the application
# How to access admin interfaces
# How to test main functionality
```

Running Tests

Shell

```
# Command to run all tests
# Command to run module-specific tests
# Any test-specific setup needed
```

Architecture Overview

- Brief explanation of your module structure
- How you solved cross-module communication
- Any additional modules or patterns are implemented

Important: Follow-up Technical Discussion

After submitting your code, you will participate in a **live technical session** where you will:

- **Walk through your code** and explain architectural decisions
- **Demonstrate deep understanding** of every part of your implementation
- **Discuss alternative approaches** and trade-offs you considered
- **Answer detailed questions** about your design patterns and framework usage
- **Show ownership** of all code submitted

Critical Requirement: You must be able to thoroughly explain and defend every aspect of your solution. This includes understanding why you chose specific patterns, how your cross-module communication works, and being able to modify or extend your code in real-time during the discussion.

Submission Guidelines

Repository Structure

- Create a public GitHub repository
- Include all source code and documentation
- Provide a clear commit history
- Tag final submission as `v1.0`

What to Submit

1. **Repository URL** with complete code
2. **Brief explanation** of your architectural approach (can be in README)
3. **Demo instructions** for testing the functionality

Bonus: Professional Development Workflow (Highly Recommended)

GitHub Actions CI/CD

Setting up automated testing and code quality checks will significantly enhance your submission. Research GitHub Actions documentation and Laravel CI/CD best practices to implement:

- Automated testing on push/pull requests (Pest)
- Code style validation (Laravel Duster)
- Static analysis (Larastan) - if implemented

Before Submission

- Application runs without errors
- All tests pass (Pest)
- Code style checks pass (Laravel Duster)
- README instructions are clear and complete
- Admin interfaces are accessible
- **BONUS:** Static analysis (Larastan) implemented and passing
- **BONUS:** GitHub Actions workflow configured

AI Tool Usage Guidelines

You are welcome to use AI tools, including agents (Claude Code is our choice, etc.) during this assignment, as this reflects modern development practices. However:

- **Verify Current Patterns:** AI training data may contain outdated framework patterns - validate against current documentation
- **Adapt and Understand:** Modify AI suggestions to fit your architectural vision and ensure you understand the code
- **Quality over Speed:** Focus on clean, maintainable solutions rather than quick implementations

Additional Guidelines

- Pay attention to versions and patterns
- Focus on clean architecture over complex features
- Document your thought process and decisions
- Test your setup instructions on a fresh environment

Good luck!